

LAB_4

October 23, 2025

```
[ ]: import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense, TimeDistributed, Input
from sklearn.model_selection import train_test_split
import warnings

tf.get_logger().setLevel('ERROR')
warnings.filterwarnings("ignore", category=UserWarning)

# --- 1. PARAMETRY I FUNKCJE POMOCNICZE ---

BITS = 20
SAMPLES = 50000

np.random.seed(42)
tf.random.set_seed(42)

def int_to_bin_array(number, bits):
    """Konwertuje liczbę całkowitą na tablicę bitów (0 lub 1), LSB na początku.
    ↪ """
    number = number % (2**bits)
    binary_str = format(number, f'0{bits}b')
    return np.array([int(bit) for bit in reversed(binary_str)])

def generate_subtraction_data(samples, bits):
    """Generuje pary liczb (A, B) i ich różnicę (A - B), zakładając A >= B."""
    X = []
    Y = []
    max_val = 2**bits - 1

    A_int = np.random.randint(0, max_val + 1, samples)
    B_int = np.random.randint(0, max_val + 1, samples)

    # Trening na nieujemnych wynikach (A >= B)
    A_int, B_int = np.maximum(A_int, B_int), np.minimum(A_int, B_int)
```

```

R_int = A_int - B_int

for a, b, r in zip(A_int, B_int, R_int):
    a_bin = int_to_bin_array(a, bits)
    b_bin = int_to_bin_array(b, bits)
    r_bin = int_to_bin_array(r, bits)

    # Wejście w każdym kroku: (bit A, bit B). Kształt: (BITS, 2)
    X.append(np.stack([a_bin, b_bin], axis=-1))

    # Wyjście: (bit Wyniku). Kształt: (BITS, 1)
    Y.append(r_bin.reshape(-1, 1))

return np.array(X), np.array(Y)

def predict_subtraction(model, a_int, b_int, bits):
    """Przygotowuje dane, uruchamia predykcję i konwertuje wynik do liczby
    całkowitej."""

    a = max(a_int, b_int)
    b = min(a_int, b_int)
    correct_int = a - b

    a_bin = int_to_bin_array(a, bits)
    b_bin = int_to_bin_array(b, bits)

    # Kształt predykcji: (1, BITS, 2)
    X_pred = np.stack([a_bin, b_bin], axis=-1)[np.newaxis, ...]

    # Predykcja
    Y_pred_prob = model.predict(X_pred, verbose=0)

    # Konwersja na bity (0 lub 1)
    Y_pred_bin = np.round(Y_pred_prob).astype(int).flatten()

    # Konwersja z powrotem na liczbę całkowitą
    result_bin_str = "".join(map(str, reversed(Y_pred_bin)))
    result_int = int(result_bin_str, 2)

    print(f"\n--- TEST PRZYKŁADU: {a} - {b} ---")
    print(f"Prawidłowy wynik: {correct_int}")
    print(f"Wynik RNN: {result_int}")
    print(f"Czy wynik jest poprawny? {result_int == correct_int}")
    print(f"Prawidłowy binarnie (MSB->LSB): {format(correct_int, f'0{BITS}b')}")
    print(f"Wynik RNN binarnie (MSB->LSB): {result_bin_str}")

# --- 2. GENEROWANIE I PODZIAŁ DANYCH ---

```

```

print("Generowanie danych treningowych...")
X, Y = generate_subtraction_data(SAMPLES, BITS)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.1,
    ↪random_state=42)
print(f"Zakończono. Kształt danych treningowych X: {X_train.shape}")

# --- 3. BUDOWA MODELU RNN (Poprawiona definicja Input Shape) ---

# Warstwa Input, aby uniknąć ostrzeżenia Keras
input_sequence = Input(shape=(BITS, 2))

model = Sequential([
    # Warstwa wejściowa definiuje kształt danych
    input_sequence,

    # SimpleRNN: Główna warstwa rekurencyjna
    SimpleRNN(units=10,
        return_sequences=True),

    # TimeDistributed: Zastosowanie gęstej warstwy do każdego kroku
    TimeDistributed(Dense(units=1, activation='sigmoid'))
])

model.compile(loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy'])

print("\n--- ARCHITEKTURA MODELU RNN ---")
model.summary()

# --- 4. TRENOWANIE MODELU ---

print("\n--- ROZPOCZĘCIE TRENINGU (ok. 20 epok) ---")
history = model.fit(X_train, Y_train,
    epochs=20,
    batch_size=128,
    validation_data=(X_test, Y_test),
    verbose=1)

print("\n Trening zakończony.")

# --- 5. OCENA I TESTOWANIE ---

loss, accuracy = model.evaluate(X_test, Y_test, verbose=0)
print(f"\n--- OCENA NA ZBIORZE TESTOWYM ---")
print(f"Test Loss: {loss:.4f}")

```

```

print(f"Test Accuracy (bit): {accuracy:.4f} (Poprawnie przewidziane bity)")

print("\n--- DEMONSTRACJA PRZYKŁADÓW ---")

# 1. Duże liczby
predict_subtraction(model, 987654, 123456, BITS)

# 2. Małe liczby
predict_subtraction(model, 15, 7, BITS)

# 3. Bliskie sobie liczby (dużo pożyczek)
predict_subtraction(model, 524287, 524286, BITS)

```

Generowanie danych treningowych...

Zakończono. Kształt danych treningowych X: (45000, 20, 2)

--- ARCHITEKTURA MODELU RNN ---

Model: "sequential"

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 20, 10)	130
time_distributed (TimeDistributed)	(None, 20, 1)	11

Total params: 141 (564.00 B)

Trainable params: 141 (564.00 B)

Non-trainable params: 0 (0.00 B)

--- ROZPOCZĘCIE TRENINGU (ok. 20 epok) ---

Epoch 1/20

352/352 2s 3ms/step -

accuracy: 0.5061 - loss: 0.7015 - val_accuracy: 0.5119 - val_loss: 0.6913

Epoch 2/20

352/352 1s 3ms/step -

accuracy: 0.5227 - loss: 0.6895 - val_accuracy: 0.5427 - val_loss: 0.6871

Epoch 3/20

352/352 1s 3ms/step -

accuracy: 0.5406 - loss: 0.6777 - val_accuracy: 0.6066 - val_loss: 0.6565

Epoch 4/20
 352/352 1s 3ms/step -
 accuracy: 0.7543 - loss: 0.5717 - val_accuracy: 0.8752 - val_loss: 0.4478

Epoch 5/20
 352/352 1s 3ms/step -
 accuracy: 0.9336 - loss: 0.3345 - val_accuracy: 0.9873 - val_loss: 0.2398

Epoch 6/20
 352/352 1s 3ms/step -
 accuracy: 0.9982 - loss: 0.1645 - val_accuracy: 1.0000 - val_loss: 0.1024

Epoch 7/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0709 - val_accuracy: 1.0000 - val_loss: 0.0489

Epoch 8/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0372 - val_accuracy: 1.0000 - val_loss: 0.0283

Epoch 9/20
 352/352 1s 2ms/step -
 accuracy: 1.0000 - loss: 0.0226 - val_accuracy: 1.0000 - val_loss: 0.0181

Epoch 10/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0149 - val_accuracy: 1.0000 - val_loss: 0.0123

Epoch 11/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0105 - val_accuracy: 1.0000 - val_loss: 0.0089

Epoch 12/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0076 - val_accuracy: 1.0000 - val_loss: 0.0066

Epoch 13/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0057 - val_accuracy: 1.0000 - val_loss: 0.0050

Epoch 14/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0044 - val_accuracy: 1.0000 - val_loss: 0.0039

Epoch 15/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0035 - val_accuracy: 1.0000 - val_loss: 0.0031

Epoch 16/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0027 - val_accuracy: 1.0000 - val_loss: 0.0024

Epoch 17/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0022 - val_accuracy: 1.0000 - val_loss: 0.0020

Epoch 18/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0018 - val_accuracy: 1.0000 - val_loss: 0.0016

Epoch 19/20
 352/352 1s 3ms/step -
 accuracy: 1.0000 - loss: 0.0014 - val_accuracy: 1.0000 - val_loss: 0.0013

Epoch 20/20
352/352 1s 3ms/step -
accuracy: 1.0000 - loss: 0.0012 - val_accuracy: 1.0000 - val_loss: 0.0011

Trening zakończony.

--- OCENA NA ZBIORZE TESTOWYM ---

Test Loss: 0.0011

Test Accuracy (bit): 1.0000 (Poprawnie przewidziane bity)

--- DEMONSTRACJA PRZYKŁADÓW ---

--- TEST PRZYKŁADU: 987654 - 123456 ---

Prawidłowy wynik: 864198

Wynik RNN: 864198

Czy wynik jest poprawny? True

Prawidłowy binarnie (MSB->LSB): 11010010111111000110

Wynik RNN binarnie (MSB->LSB): 11010010111111000110

--- TEST PRZYKŁADU: 15 - 7 ---

Prawidłowy wynik: 8

Wynik RNN: 8

Czy wynik jest poprawny? True

Prawidłowy binarnie (MSB->LSB): 00000000000000001000

Wynik RNN binarnie (MSB->LSB): 00000000000000001000

--- TEST PRZYKŁADU: 524287 - 524286 ---

Prawidłowy wynik: 1

Wynik RNN: 1

Czy wynik jest poprawny? True

Prawidłowy binarnie (MSB->LSB): 00000000000000000001

Wynik RNN binarnie (MSB->LSB): 00000000000000000001